

Creational Design Patterns

Instructor: Muhammad Zeeshan Nazar

Creational Design Patterns

- Creational design patterns provide various **object creation** mechanisms, which increase **flexibility** and **reuse** of existing code.
- Creational design patterns **abstract** the instantiation process.
- They help make a system independent of how its objects are created, composed, and represented.
- A **class creational pattern** uses inheritance to vary the class that's instantiated.
- An **object creational pattern** will delegate instantiation to another object.

Creational Design Patterns

- First, they all encapsulate knowledge about which concrete classes the system uses.
- Second, they hide how instances of these classes are created and put together.
- Creational patterns give you a lot of flexibility in what gets created, who creates it, how it gets created, and when.

Catalog of Design Patterns

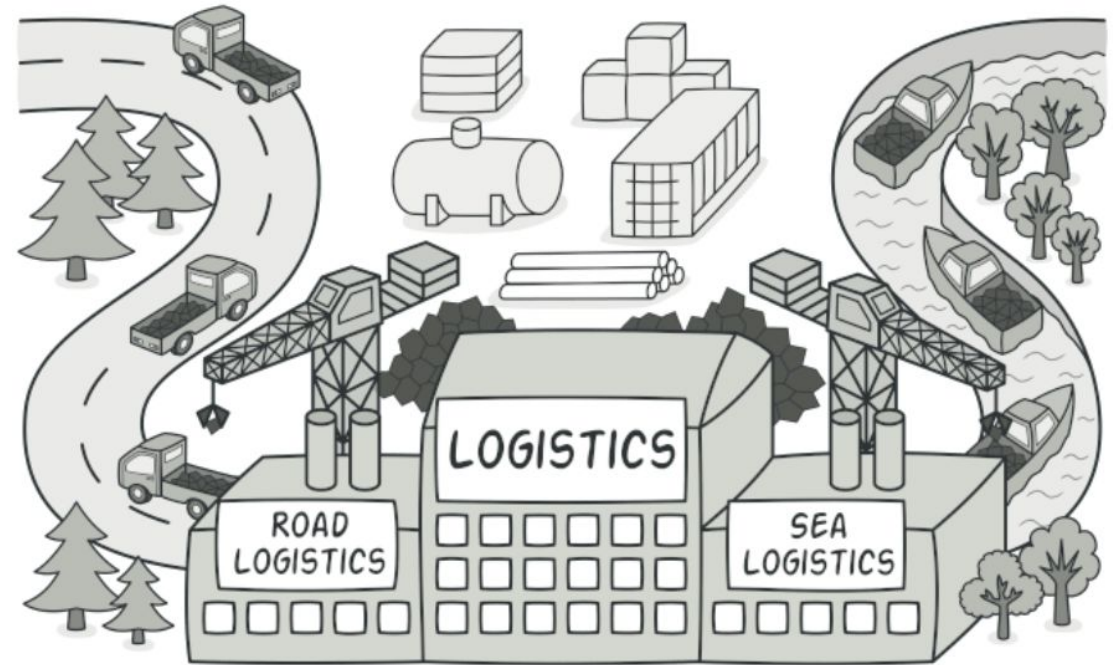
		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method (107)	Adapter (class) (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (object) (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Flyweight (195) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Observer (293) State (305) Strategy (315) Visitor (331)

Factory Method Pattern

Factory Method

Intent:

- **Factory Method** is a creational design pattern that provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created.
- It is also known as **Virtual Constructor**



Problem

- Imagine that you're creating a logistics management application. The first version of your app can only handle transportation by trucks, so the bulk of your code lives inside the Truck class.
- After a while, your app becomes popular. Each day you receive dozens of requests from sea transportation companies to incorporate sea logistics into the app.



Adding a new class to the program isn't that simple if the rest of the code is already coupled to existing classes.

Problem

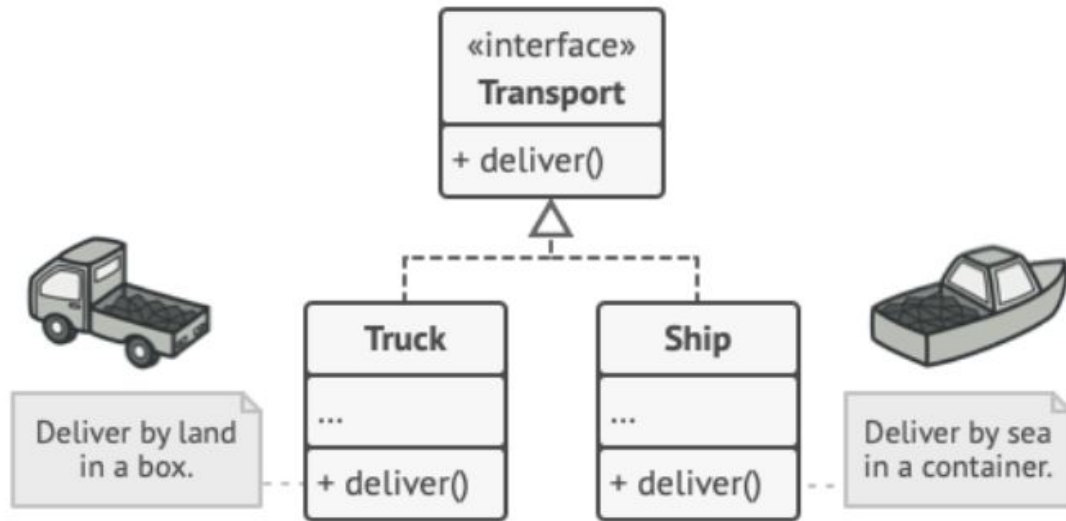
- At present, most of your code is coupled to the Truck class. Adding Ships into the app would require making changes to the entire codebase.
- Moreover, if later you decide to add another type of transportation to the app, you will probably need to make all these changes again.
- As a result, you will end up with nasty code, riddled with conditionals that switch the app's behavior depending on the class of transportation objects.



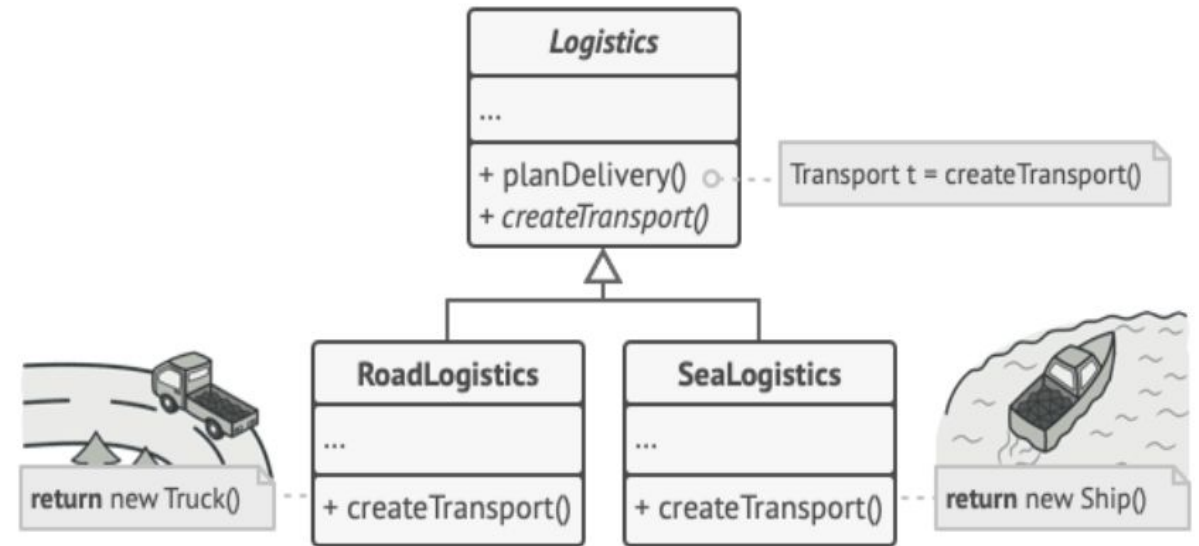
Adding a new class to the program isn't that simple if the rest of the code is already coupled to existing classes.

Solution

- The Factory Method pattern suggests that you replace direct object construction calls (using the new operator) with calls to a special factory method.
- Objects are still created, but it's being called from within the factory method.
- Objects returned by a factory method are often referred to as **products**.



All products must follow the same interface.



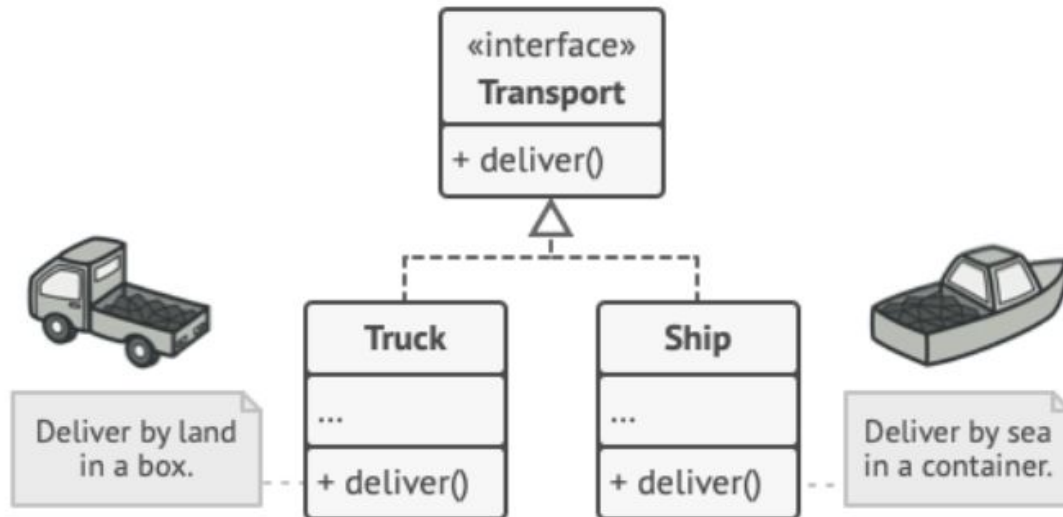
Subclasses can alter the class of objects being returned by the factory method.

Solution

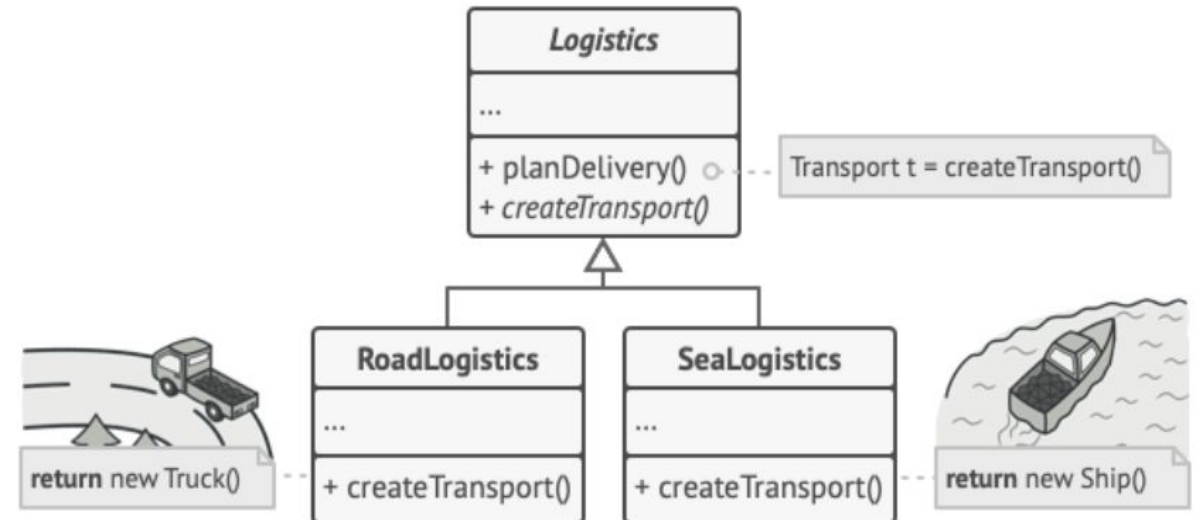
- It looks like we just moved new Truck() or new Ship() from the main program into another function (the factory). So it may seem like nothing special happened.
- Now we can change what kind of object is created in child classes without touching old code.
 - **For example:**
 - RoadLogistics creates a Truck
 - SeaLogistics creates a Ship
 - Both use the same createTransport() method, but each makes a different product. This gives flexibility..
- All the products (like Truck, Ship, Plane) must share a common parent — for example, Transport. So that the factory can return them through one type.

Transport* createTransport(); // returns any transport

- The base class's factory method must return the common type (like Transport), so all subclasses can return their own products easily.



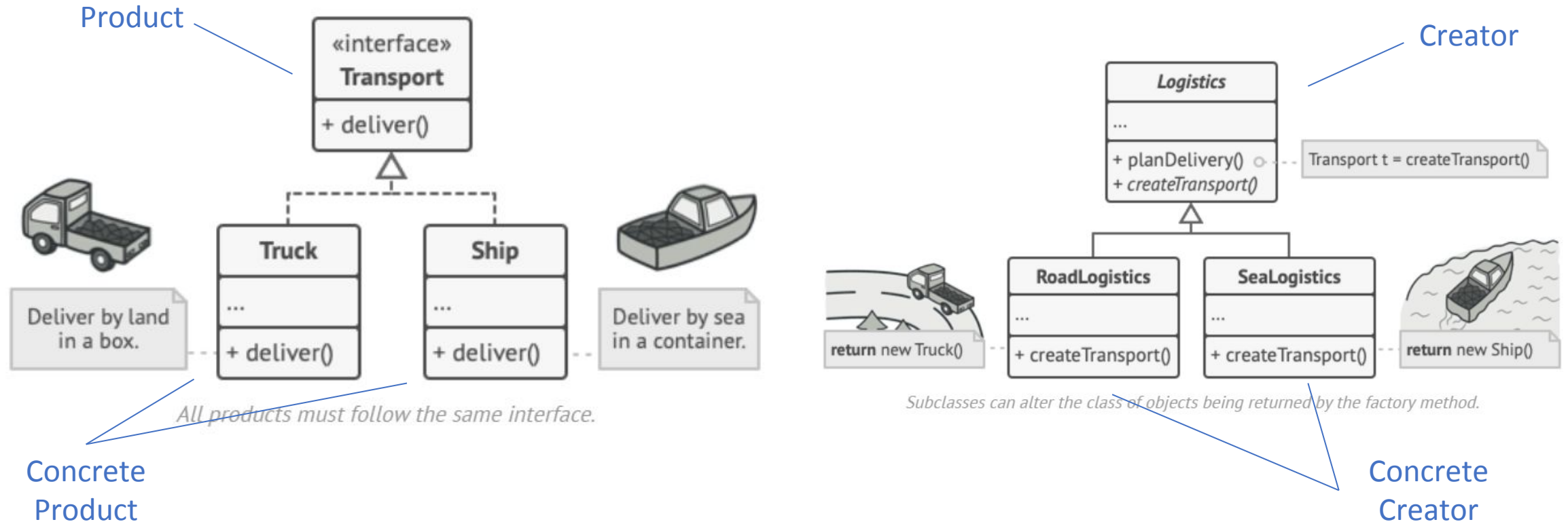
All products must follow the same interface.



Subclasses can alter the class of objects being returned by the factory method.

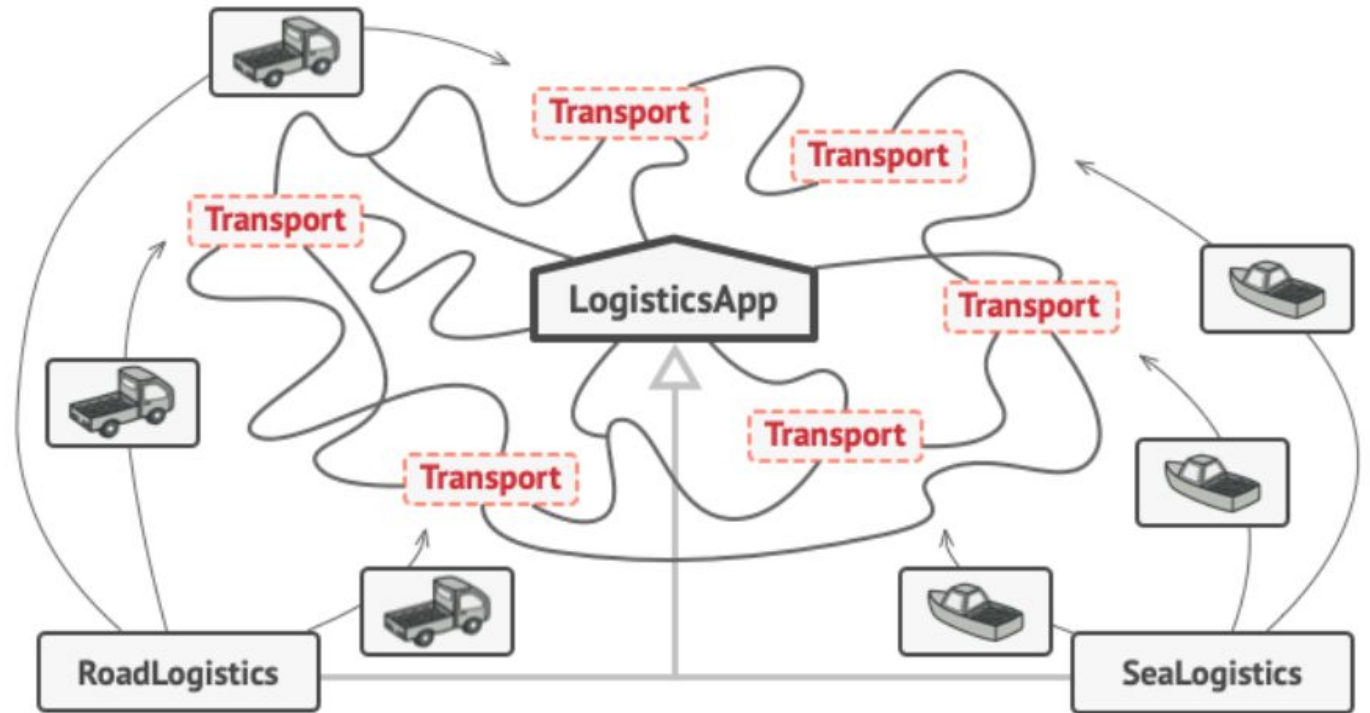
Solution

- For example, both Truck and Ship classes should implement the Transport interface, which declares a method called deliver.
- Each class implements this method differently: trucks deliver cargo by land and ships deliver cargo by sea.
- The factory method in the RoadLogistics class returns truck objects, whereas the factory method in the SeaLogistics class returns ships.



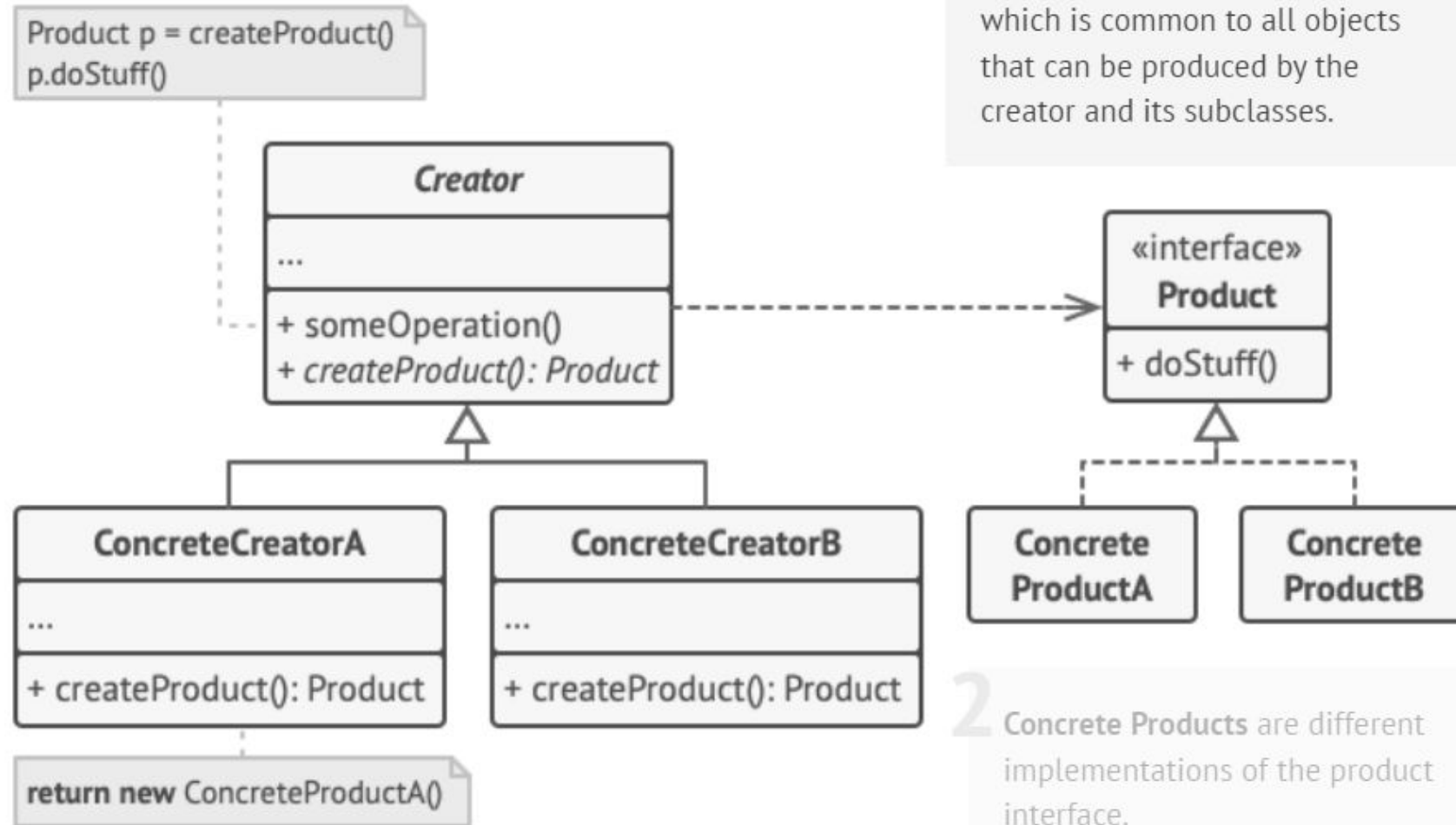
Solution

- The code that uses the factory method (often called the **client code**) doesn't see a difference between the actual products returned by various subclasses.
- The client treats all the products as abstract Transport.
- The client knows that all transport objects are supposed to have the deliver method, but exactly how it works isn't important to the client.

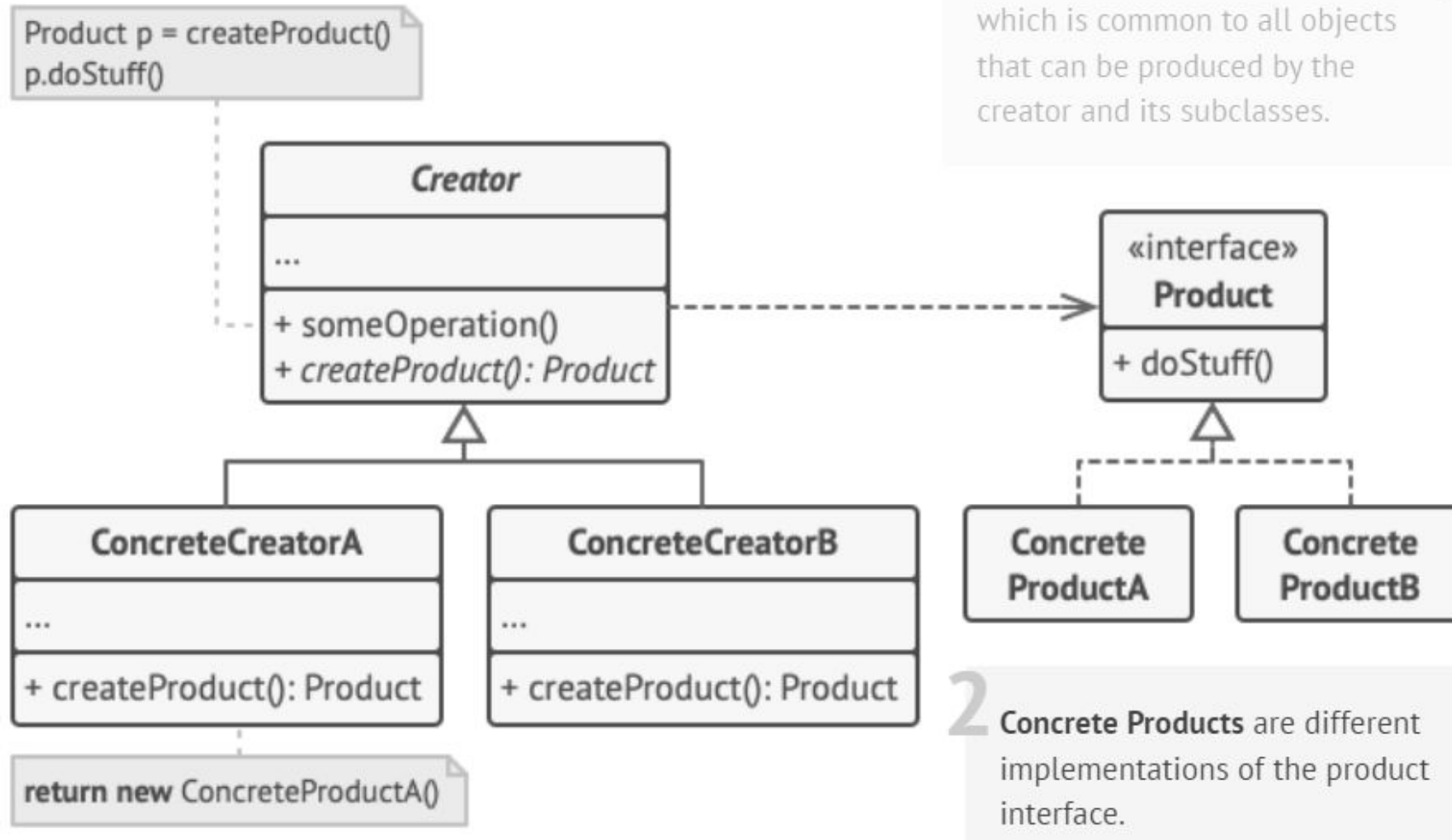


As long as all product classes implement a common interface, you can pass their objects to the client code without breaking it.

Structure



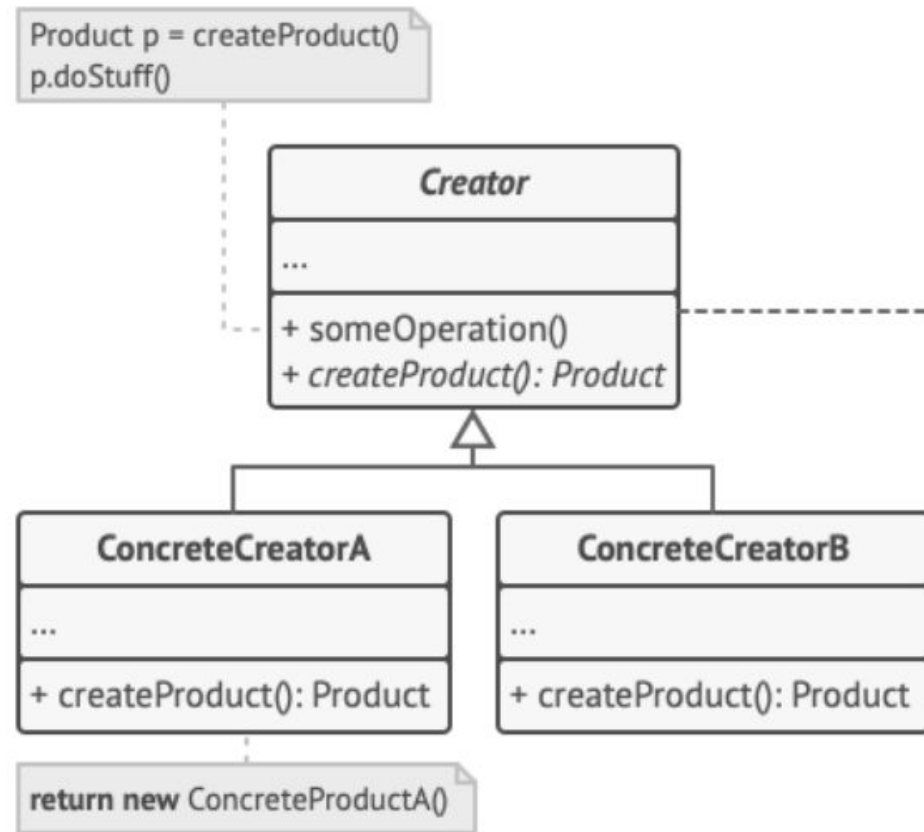
Structure



3 The **Creator** class declares the factory method that returns new product objects. It's important that the return type of this method matches the product interface.

You can declare the factory method as `abstract` to force all subclasses to implement their own versions of the method. As an alternative, the base factory method can return some default product type.

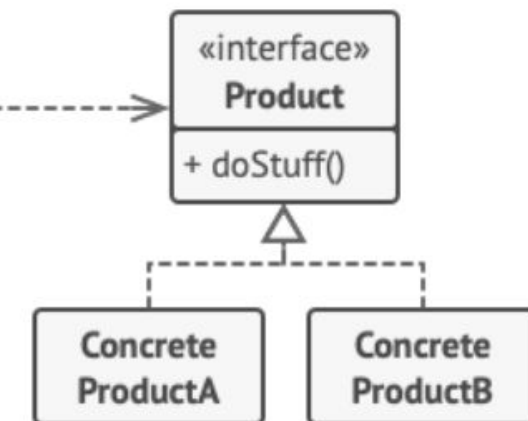
Note, despite its name, product creation is **not** the primary responsibility of the creator. Usually, the creator class already has some core business logic related to products. The factory method helps to decouple this logic from the concrete product classes. Here is an analogy: a large software development company can have a training department for programmers. However, the primary function of the company as a whole is still writing code, not producing programmers.



4 **Concrete Creators** override the base factory method so it returns a different type of product.

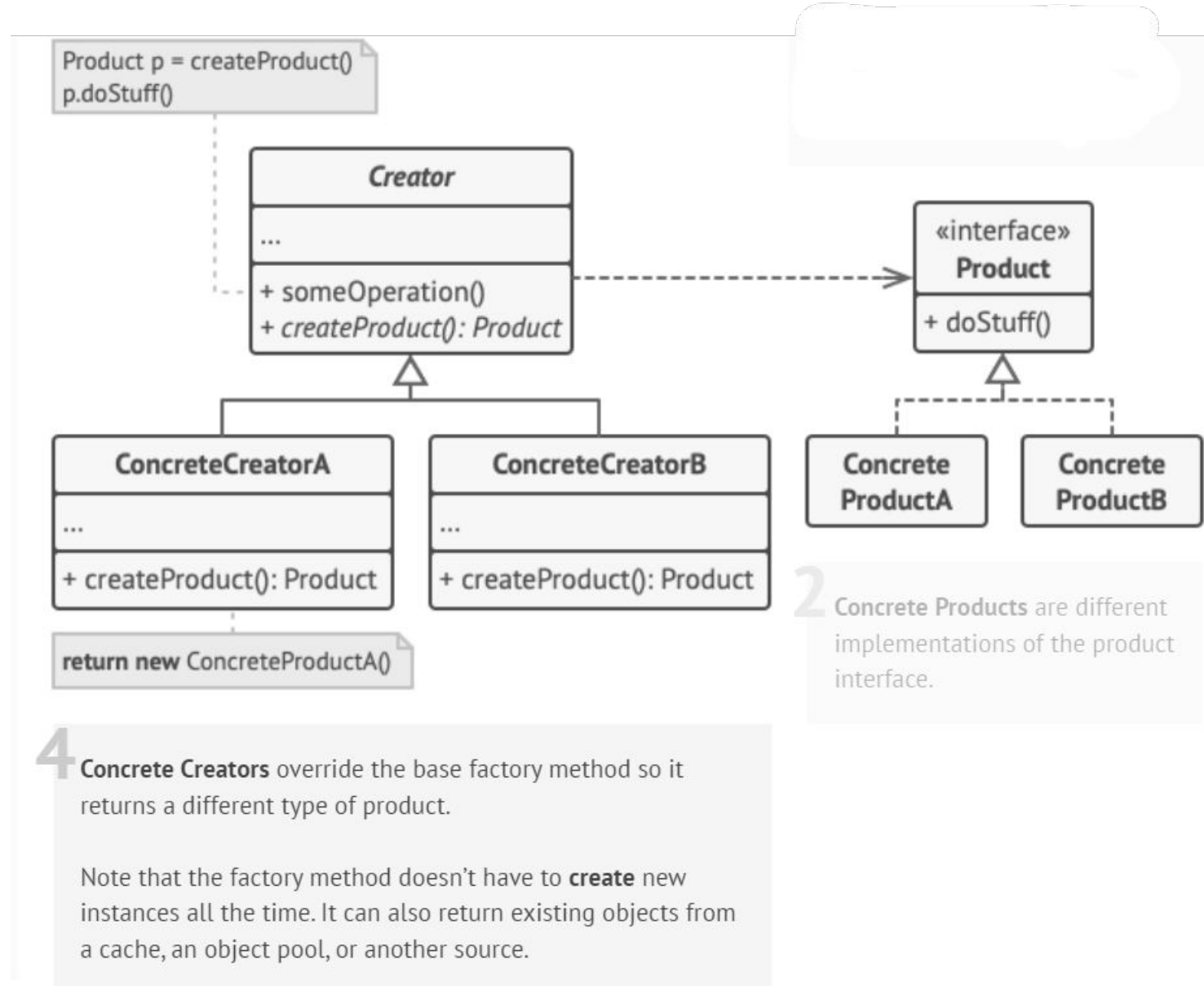
Note that the factory method doesn't have to **create** new instances all the time. It can also return existing objects from a cache, an object pool, or another source.

1 The **Product** declares the interface, which is common to all objects that can be produced by the creator and its subclasses.



2 **Concrete Products** are different implementations of the product interface.

Structure



Example

```
class Pizza {
public:
    void prepare() const {
        cout << "Preparing a pizza...\n";
    }
};

class Pasta {
public:
    void prepare() const {
        cout << "Preparing a pasta...\n";
    }
};

class Waiter {
public:
    void takeOrder(const string& dishType) {
        if (dishType == "Pizza") {
            Pizza pizza;
            pizza.prepare();
        } else if (dishType == "Pasta") {
            Pasta pasta;
            pasta.prepare();
        } else {
            cout << "Sorry, we don't serve that dish.\n";
        }
    }
};
```

Example

```
int main() {  
    Waiter waiter;  
    waiter.takeOrder("Pizza");  
    waiter.takeOrder("Pasta");  
    waiter.takeOrder("Burger");  
  
    return 0;  
}
```

Issues

- **Tight Coupling:** Waiter directly depends on Pizza and Pasta classes, making it difficult to add new dishes without modifying Waiter.
- **Violation of Open/Closed Principle:** New dish types require modification in Waiter

Example (Applying Factory Method Pattern)

```
// Dish Interface
class Dish {
public:
    virtual void prepare() const = 0;
};

// Concrete Dishes
class Pizza : public Dish {
public:
    void prepare() const override {
        cout << "Preparing a pizza...\n";
    }
};

class Pasta : public Dish {
public:
    void prepare() const override {
        cout << "Preparing a pasta...\n";
    }
};
```

Example (Applying Factory Method Pattern)

```
// Chef Interface
class Chef {
public:
    virtual Dish* createDish() const = 0;
};

// Concrete Chefs
class PizzaChef : public Chef {
public:
    Dish* createDish() const override {
        return new Pizza();
    }
};

class PastaChef : public Chef {
public:
    Dish* createDish() const override {
        return new Pasta();
    }
};
```

Example (Applying Factory Method Pattern)

```
// Waiter function that uses Chef objects
void waiterTakesOrder(const Chef& chef) {
    Dish* dish = chef.createDish();
    dish->prepare();
}

int main() {
    PizzaChef pizzaChef;
    PastaChef pastaChef;

    cout << "Waiter taking order for pizza:\n";
    waiterTakesOrder(pizzaChef);

    cout << "Waiter taking order for pasta:\n";
    waiterTakesOrder(pastaChef);

    return 0;
}
```

Advantages

- **Reduced Coupling:** Waiter no longer needs to know about specific dishes like Pizza or Pasta. It only interacts with the Chef interface, which makes it easy to add new dishes without modifying Waiter.
- **Scalability:** To add a new dish (e.g., Burger), simply create a new Burger class and BurgerChef class without changing the Waiter.

Example

```
// Library classes
```

```
class Vehicle {
```

```
public:
```

```
    virtual void printVehicle() = 0;
```

```
};
```

```
class TwoWheeler : public Vehicle {
```

```
public:
```

```
    void printVehicle()
```

```
    {
```

```
        cout << "I am two wheeler" << endl;
```

```
    }
```

```
};
```

```
class FourWheeler : public Vehicle {
```

```
public:
```

```
    void printVehicle()
```

```
    {
```

```
        cout << "I am four wheeler" << endl;
```

```
    }
```

```
};
```

```
// Client (or user) class
```

```
class Client {
```

```
public:
```

```
    Client(int type)
```

```
    {
```

```
        // Client explicitly creates classes according to type
```

```
        if (type == 1)
```

```
            pVehicle = new TwoWheeler();
```

```
        else if (type == 2)
```

```
            pVehicle = new FourWheeler();
```

```
        else
```

```
            pVehicle = NULL;
```

```
    }
```

```
    Vehicle* getVehicle() { return pVehicle; }
```

```
private:
```

```
    Vehicle* pVehicle;
```

```
};
```

Example

```
// Driver program
int main()
{
    Client* pClient = new Client(1);
    Vehicle* pVehicle = pClient->getVehicle();
    pVehicle->printVehicle();
    return 0;
}
//Output: I am two wheeler
```


Problem with this design?

What are the problems with the above design?

- As you must have observed in the above example, the Client creates objects of either TwoWheeler or FourWheeler based on some input during the construction of its object.
- Say, the library introduces a new class ThreeWheeler to incorporate three-wheeler vehicles also.

What would happen?

- The client will end up chaining a new else if in the conditional ladder to create objects of ThreeWheeler. Which in turn will need the Client to be recompiled.
- So, each time a new change is made on the library side, the Client would need to make some corresponding changes at its end and recompile the code.
- Sounds bad? This is a very bad practice of design.

How do we avoid the problem?

- The answer is, to create a vehicle's factory class with a single responsibility to create the objects.

Example

```
enum VehicleType {
    VT_TwoWheeler,
    VT_ThreeWheeler,
    VT_FourWheeler
};

// Library classes

class Vehicle {
public:
    virtual void printVehicleInfo() = 0;
};

class TwoWheeler : public Vehicle {
public:
    void printVehicleInfo()
    {
        cout << "I am two wheeler" << endl;
    }
};
```

```
class ThreeWheeler : public Vehicle {
public:
    void printVehicleInfo()
    {
        cout << "I am three wheeler" << endl;
    }
};

class FourWheeler : public Vehicle {
public:
    void printVehicleInfo()
    {
        cout << "I am four wheeler" << endl;
    }
};
```

Example

// Class Vehicle Factory with a single responsibility to construct Vehicles according to the clients' requests.

```
class VehicleFactory {  
public:  
    Vehicle* build(VehicleType vehicleType)  
    {  
        if (vehicleType == VT_TwoWheeler)  
            return new TwoWheeler();  
        else if (vehicleType == VT_ThreeWheeler)  
            return new ThreeWheeler();  
        else if (vehicleType == VT_FourWheeler)  
            return new FourWheeler();  
        else  
            return nullptr;  
    }  
};
```

// Client class. The client object will ask the factory to build vehicles.

```
class Client {  
public:  
    Client() {  
        pVehicle = nullptr;  
    }  
    void BuildVehicle(VehicleType vehicleType)  
    {  
        VehicleFactory* vf = new VehicleFactory();  
        pVehicle = vf->build(vehicleType);  
    }  
    Vehicle* getVehicle() { return pVehicle; }  
  
private:  
    Vehicle* pVehicle;  
};
```

Example

```
// Driver program
```

```
int main()
```

```
{
```

```
    Client* pClient = new Client();
```

```
    pClient->BuildVehicle(VT_TwoWheeler);
```

```
    pClient->getVehicle()->printVehicleInfo();
```

```
    pClient->BuildVehicle(VT_ThreeWheeler);
```

```
    pClient->getVehicle()->printVehicleInfo();
```

```
    pClient->BuildVehicle(VT_FourWheeler);
```

```
    pClient->getVehicle()->printVehicleInfo();
```

```
}
```

```
//Output
```

```
I am two wheeler
```

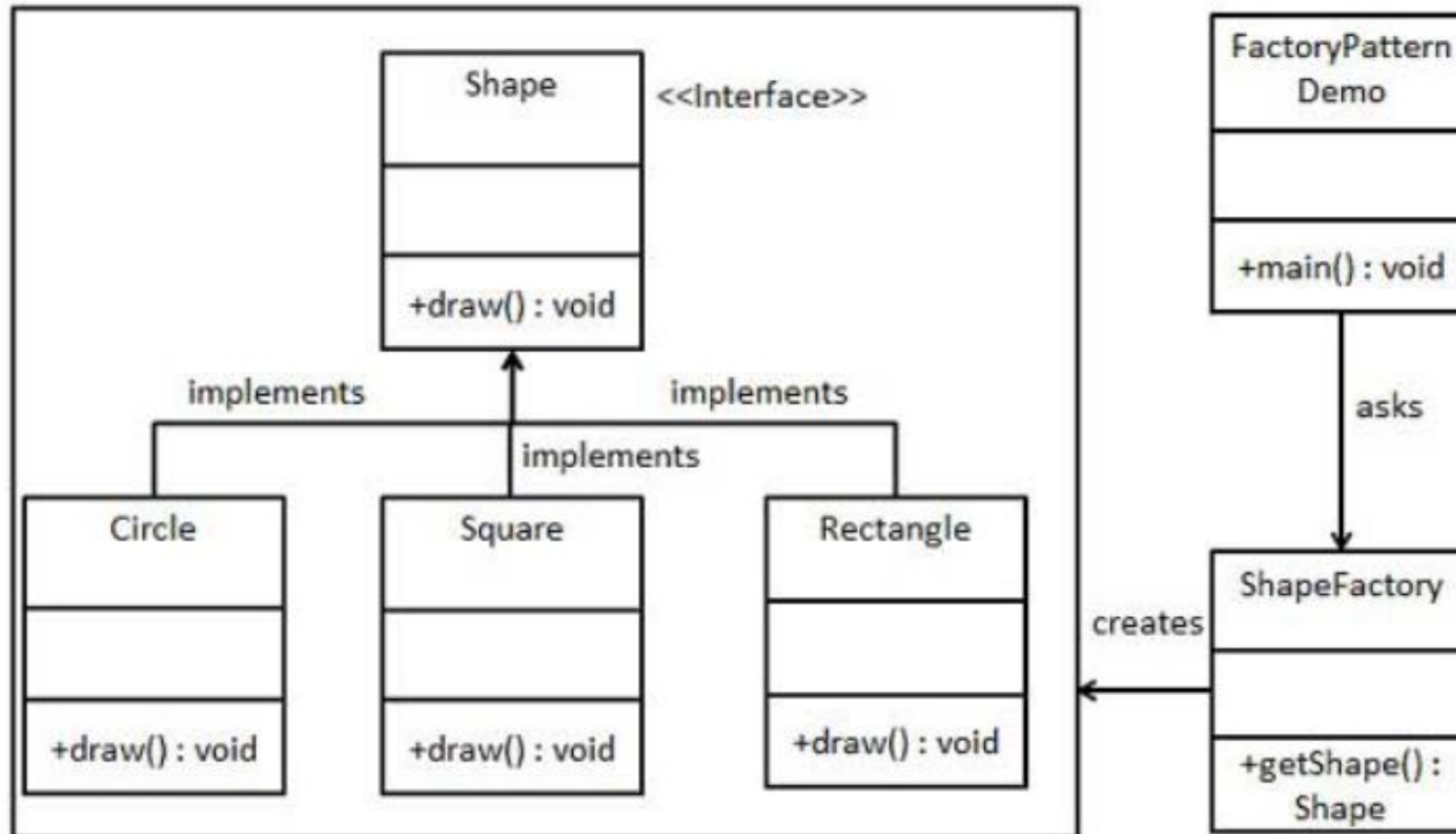
```
I am three wheeler
```

```
I am four wheeler
```

Example

- In the above example, we have totally decoupled the selection of types for object creation from the Client.
- The factory class is now responsible for deciding which object type to create based on the input provided by the client.
- The client just needs to make calls to the factory class's build method and pass the type it wants without worrying about the actual implementation of the creation of objects.

Another Example



Example

// Step 1: Create interface

```
public interface Shape {  
    void draw();  
}
```

// Step 2: Create concrete classes implementing the interface

```
public class Rectangle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Inside Rectangle::draw() method.");  
    }  
}  
  
public class Square implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Inside Square::draw() method.");  
    }  
}
```

```
public class Circle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Inside Circle::draw() method.");  
    }  
}
```

Example

// Step 3: Create a Factory to generate object of concrete class based on given information.

```
public class ShapeFactory {  
    //use getShape method to get object of type shape  
    public Shape getShape(String shapeType){  
        if(shapeType == null){  
            return null;  
        }  
        if(shapeType.equalsIgnoreCase("CIRCLE")){  
            return new Circle();  
        } else if(shapeType.equalsIgnoreCase("RECTANGLE")){  
            return new Rectangle();  
        } else if(shapeType.equalsIgnoreCase("SQUARE")){  
            return new Square();  
        }  
        return null;  
    }  
}
```

// Step 4: Use the Factory to get object of concrete class by passing an information such as type.

```
public class FactoryPatternDemo {  
    public static void main(String[] args) {  
        ShapeFactory shapeFactory = new ShapeFactory();  
        //get an object of Circle and call its draw method.  
        Shape shape1 = shapeFactory.getShape("CIRCLE");  
        //call draw method of Circle  
        shape1.draw();  
        //get an object of Rectangle and call its draw method.  
        Shape shape2 = shapeFactory.getShape("RECTANGLE");  
        //call draw method of Rectangle  
        shape2.draw();  
        //get an object of Square and call its draw method.  
        Shape shape3 = shapeFactory.getShape("SQUARE");  
        //call draw method of square  
        shape3.draw();  
    }  
}
```


Example

// Step 4: Use the Factory to get object of concrete class by passing an information such as type.

```
public class FactoryPatternDemo {  
    public static void main(String[] args) {  
        ShapeFactory shapeFactory = new ShapeFactory();  
        //get an object of Circle and call its draw method.  
        Shape shape1 = shapeFactory.getShape("CIRCLE");  
        //call draw method of Circle  
        shape1.draw();  
        //get an object of Rectangle and call its draw method.  
        Shape shape2 = shapeFactory.getShape("RECTANGLE");  
        //call draw method of Rectangle  
        shape2.draw();  
        //get an object of Square and call its draw method.  
        Shape shape3 = shapeFactory.getShape("SQUARE");  
        //call draw method of square  
        shape3.draw();  
    }  
}
```

// Output:

Inside Circle::draw() method.

Inside Rectangle::draw() method.

Inside Square::draw() method.

References

- Design Patterns: Elements of Reusable Object-Oriented Software, Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, Pearson, 1995.
- Helping Links:
- <https://refactoring.guru/design-patterns/>
- <https://www.geeksforgeeks.org/factory-method-for-designing-pattern/>
- https://sourcemaking.com/design_patterns/